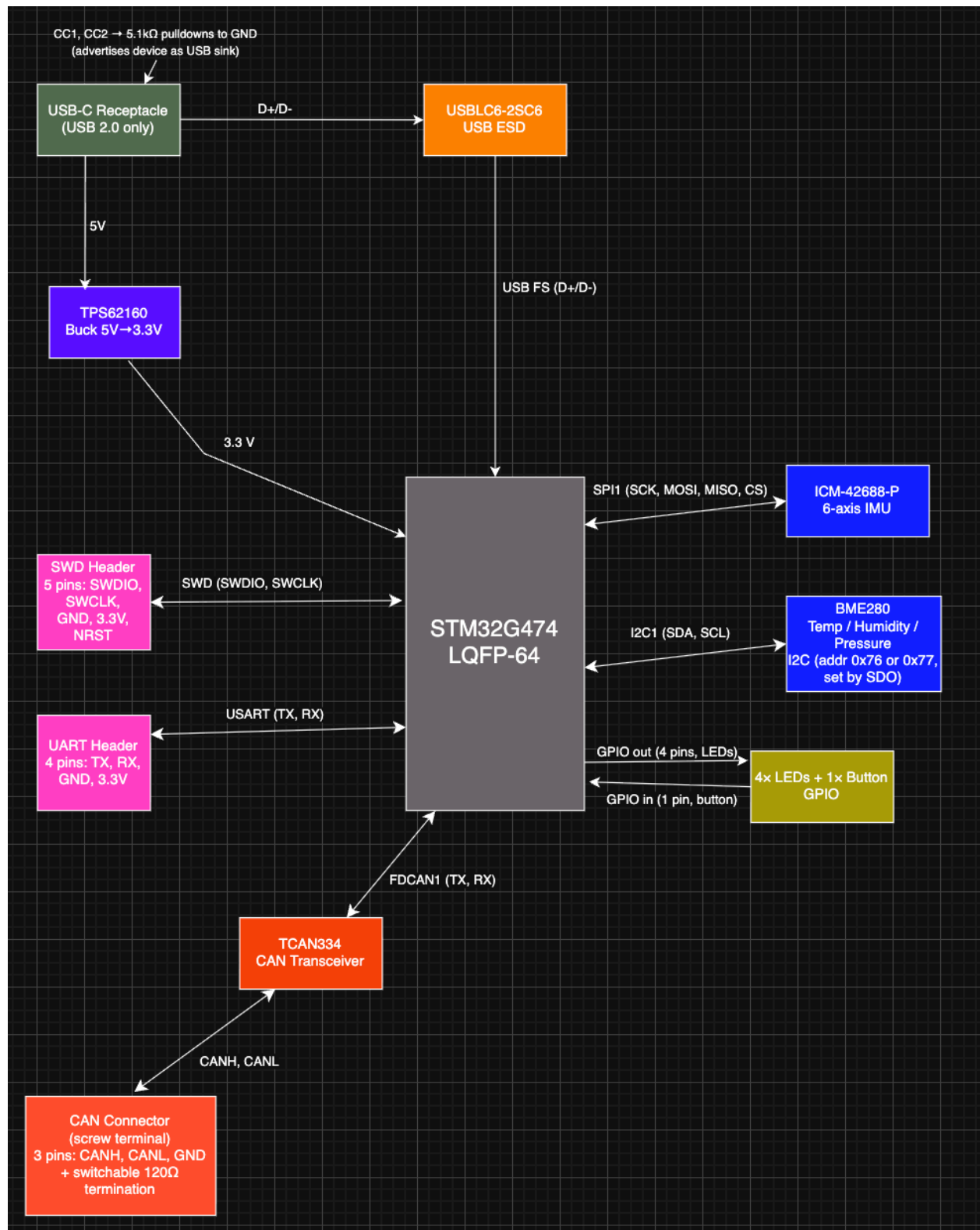
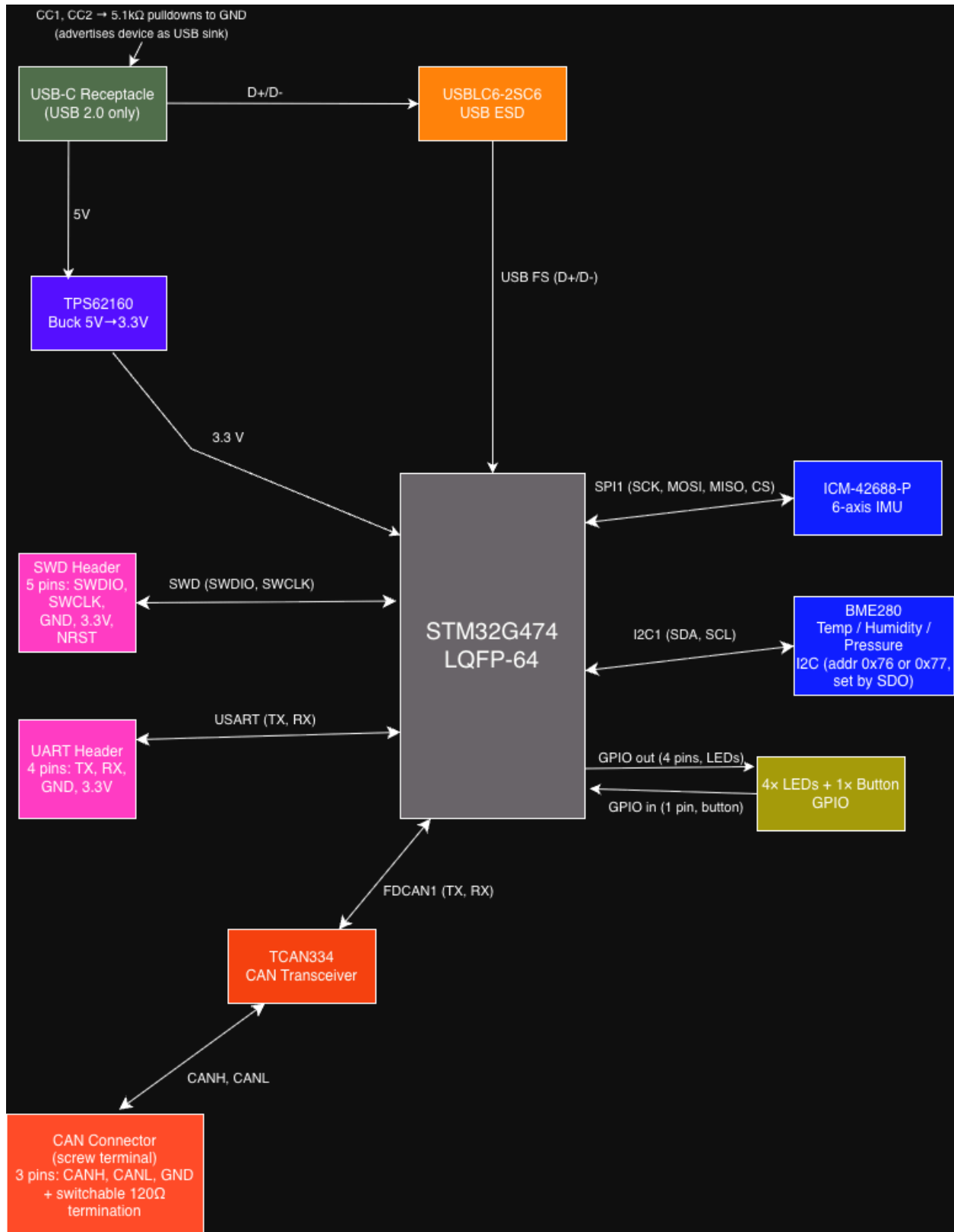






CANaryNode Block Diagram — Full Layout

Canvas organization:

- Left band: external connectors and inputs
 - Middle band: the STM32G474 MCU (hub)
 - Right band: sensors and outputs
 - Power flows top-to-bottom on the left side; signals flow left-to-right into the MCU
-

Power section (top-left, flowing down)

USB-C Receptacle (top-left corner) Outputs going down/right:

- VBUS (5V) → down to buck converter
- D+ / D- → right through USBLC6 → into MCU USB pins
- CC1, CC2 → each has a 5.1kΩ pulldown to GND (advertises "I'm a USB device, give me 5V")
- GND, Shield

TPS62160 Buck Converter (below USB-C)

- Input: 5V from USB VBUS
- Output: 3.3V rail that feeds *everything* — MCU, both sensors, transceiver
- This is your main power rail

Power rail color convention on the diagram:

- Red lines/labels = 5V
 - Orange lines/labels = 3.3V
 - Black/gray = GND
 - Blue = signal arrows
-

Protection / ESD (between USB-C and MCU)

USBLC6-2SC6

- Sits in the data path between USB-C connector and MCU USB pins
 - Clamps ESD spikes on D+/D-
 - 4 pins of interest: D+ in, D+ out, D- in, D- out (plus VCC, GND)
-

The MCU (center)

STM32G474 (LQFP package, TBD pin count) Label the following interfaces on the MCU block, arranged around its edges:

Left side of MCU (signals coming in from the left band):

- USB (D+/D-) — connects to USBLC6
- SWD (debug) — to SWD header
- UART (debug) — to UART header

Right side of MCU (signals going out to sensors):

- SPI1 → ICM-42688-P (IMU)
- I2C1 → BME280 (environmental)
- (Optional SPI2 or shared SPI1 with different CS → MAX31855 thermocouple)

Bottom of MCU:

- FDCAN1 (TX, RX) → TCAN334 transceiver
- GPIO × 4 → LEDs
- GPIO × 1 → user button

Sensors (right band)

ICM-42688-P (IMU, top-right)

- Connected to MCU via SPI: SCK, MOSI, MISO, CS (4 wires)
- 6-axis: 3-axis accel + 3-axis gyro
- Powered from 3.3V rail

BME280 (environmental, middle-right)

- Connected to MCU via I2C: SDA, SCL (2 wires)
- Temperature, humidity, pressure
- Powered from 3.3V rail
- I2C address: 0x76 or 0x77 (set by SDO pin)

MAX31855 (optional thermocouple amp, bottom-right — mark as optional)

- Connected to MCU via SPI (shares bus with IMU, separate CS pin)
- Reads K-type thermocouple for high temperatures

CAN section (bottom-middle)

TCAN334 CAN Transceiver

- MCU side (logic, 3.3V): TXD, RXD connect to MCU FDCAN1 pins
- Bus side (differential, ~2.5V common mode): CANH, CANL go out to connector
- Powered from 3.3V rail

CAN Connector (bottom-left corner — screw terminal or DB9)

- Pins: CANH, CANL, GND
 - Switchable 120Ω termination resistor between CANH and CANL (jumper or DIP switch)
-

Debug + user interface (left band, below USB-C)

SWD Header (5 pins)

- SWDIO, SWCLK, GND, 3.3V, NRST
- This is how you flash and debug — connects to ST-Link

UART Header (4 pins)

- TX, RX, GND, 3.3V
- Backup debug channel, printf logging

LEDs × 4 (right of MCU or bottom)

- Each driven by a GPIO through a current-limiting resistor
- Conventional assignment: power LED (always on when 3.3V present), heartbeat (toggles in main loop), CAN activity, user/fault

Button × 1

- One side to GPIO, other side to GND (active-low)
 - GPIO uses MCU internal pull-up
-

Recap of signal flow

A CAN message arrives → TCAN334 converts differential to logic → MCU FDCAN1 RX → firmware processes it.

A sensor reading happens → MCU SPI1 clocks data out of ICM-42688-P → firmware packages it → MCU FDCAN1 TX → TCAN334 drives differential bus → message goes out.

USB config: laptop sends command → USB-C → USBLC6 → MCU USB → firmware updates
config struct → response back the same path.

Two small additions to file away:

- It's not just the MCU that needs 3.3V — both sensors and the CAN transceiver also run on 3.3V. The buck feeds the whole board.
- "Why a buck and not an LDO?" → efficiency. LDO burns the extra voltage as heat (~66% efficient), buck switches it (~90% efficient). Memorize that — it's a guaranteed interview question.

USB Data Path (D+/D-) and ESD Protection

D+ and D-

- The two data wires of USB. They carry data as a *differential pair* — D+ and D- always swing in opposite directions.
- Differential signaling rejects noise: any noise that hits both wires equally gets cancelled when the receiver looks at the difference between them.
- USB Full Speed (what the G474 supports) = 12 Mbps over this pair.
- Must be routed as a controlled-impedance 90Ω differential pair on the PCB (this is why the JLC04161H-7628 stackup was chosen).
- Trace length matching matters: D+ and D- should be the same length within a few mm so signals arrive simultaneously.

Why ESD protection is needed

- ESD = electrostatic discharge. Walking across carpet and touching a USB plug can dump ~8–15 kV into the connector for a microsecond.
- USB pins on the MCU are rated for maybe ±2 kV human-body-model. Direct ESD = dead MCU.
- The USB-C connector is the most exposed part of the board — fingers and cables touch it constantly. Highest ESD risk on the whole design.

What USBLC6-2SC6 does

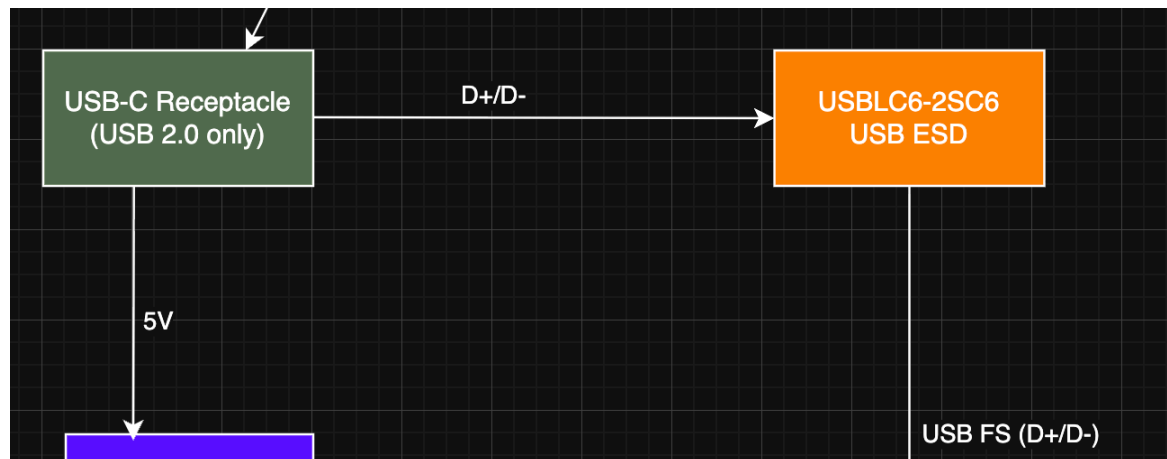
- TVS diode array (Transient Voltage Suppressor).
- Sits across each data line to ground. Normally invisible (high impedance, low capacitance ~3.5 pF so it doesn't distort USB signals).
- When an ESD spike hits, the diode "snaps on" in nanoseconds and shunts the current to ground before it reaches the MCU.
- After the spike passes, it returns to high impedance. Self-resetting.
- Standard part on every USB 2.0 design — costs \$0.08, prevents a dead MCU.

Why specifically the USBLC6-2SC6 and not something else

- Low capacitance (3.5 pF) — critical for high-speed signals. A cheap TVS with 20 pF capacitance would distort USB.
- 4-line variant: protects D+, D-, plus has integrated steering diodes to a single TVS element. Compact.
- IEC 61000-4-2 rated for ± 15 kV air discharge — way more than ESD events in the real world.

Placement rule (matters in layout, not the block diagram yet)

- USBLC6 must be placed *as close to the USB-C connector as possible* — ideally within a few mm.
- ESD energy has to be shunted *before* it reaches anything fragile. If the ESD chip is on the MCU side of a 5cm trace, the trace itself is the antenna and the MCU still sees the spike.
- This is a "rules people violate and then wonder why their board dies" thing. Worth remembering.



The problem USB-C solves: Old USB (A/B) had different connectors for hosts and devices. A laptop = host = supplies power. A phone = device = receives power. Connector shape told you which was which.

USB-C uses the *same connector on both ends*. So how does the laptop know "this is a device that wants power" vs "this is another laptop"?

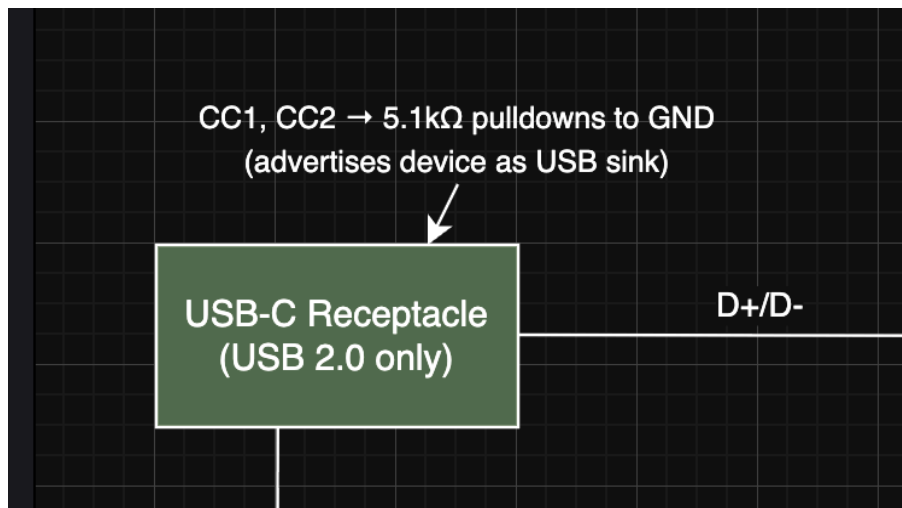
The answer: CC pins. USB-C has two pins called CC1 and CC2 (Configuration Channel). They're how the two ends negotiate roles.

- A **host** (laptop) puts **pull-up resistors** on its CC pins (5.1k Ω to 5V)
- A **device** (your board) puts **pull-down resistors** on its CC pins (5.1k Ω to GND)

When you plug in, the laptop senses the voltage on the CC line. If it sees the pulldown, it thinks "ok, that's a device, I'll turn on 5V." If it sees nothing (no pulldown), it thinks "nothing's there" and keeps VBUS off.

Why 5.1kΩ specifically: The USB-C spec defines this exact value. The laptop measures the voltage divider formed by its pull-up and your pull-down. 5.1kΩ produces the voltage the spec expects for "standard device, draw up to 3A."

What you need to do: Two 5.1kΩ resistors on your board — one from CC1 to GND, one from CC2 to GND. Two resistors because the connector is reversible — whichever orientation you plug in, one of them becomes the active CC line.



SPI = Serial Peripheral Interface. A 4-wire protocol for the MCU to talk to chips like sensors, flash memory, displays. Faster than I2C, simpler than USB.

The 4 wires:

SCK — Serial Clock. MCU generates a clock signal; every tick = one bit transferred. Whoever controls the clock controls the speed.

MOSI — Master Out, Slave In. Data from MCU → sensor. (Commands, register addresses, config.)

MISO — Master In, Slave Out. Data from sensor → MCU. (Sensor readings.)

CS — Chip Select. A "wake up, I'm talking to you" line. MCU pulls CS low to start a transaction, high to end it. Active-low.

Why CS exists: SPI lets multiple devices share SCK/MOSI/MISO. Each device gets its own CS line. Only the chip with CS pulled low listens; the rest ignore the bus. That's how you'd add the optional MAX31855 thermocouple later — same SPI bus, second CS pin.

SPI1: STM32G474 has multiple SPI peripherals (SPI1, SPI2, SPI3, SPI4). "SPI1" just names which hardware peripheral you're using. The choice affects which physical pins and which DMA channels — that's a CubeMX decision later.

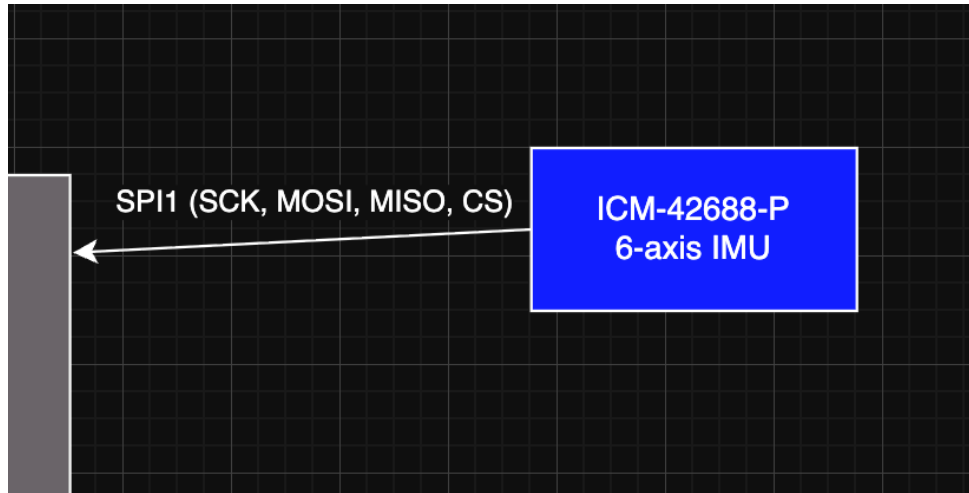
A transaction looks like:

MCU pulls CS low

MCU clocks out a register address on MOSI while reading garbage on MISO

MCU clocks dummy bytes on MOSI while reading sensor data on MISO

MCU pulls CS high



I2C = Inter-Integrated Circuit. A 2-wire protocol for short-distance, low-speed chip-to-chip communication.

The 2 wires:

SDA — Serial Data. Bidirectional, carries both addresses and data.

SCL — Serial Clock. MCU generates the clock.

Key differences from SPI:

2 wires vs 4 — much less PCB space, simpler routing.

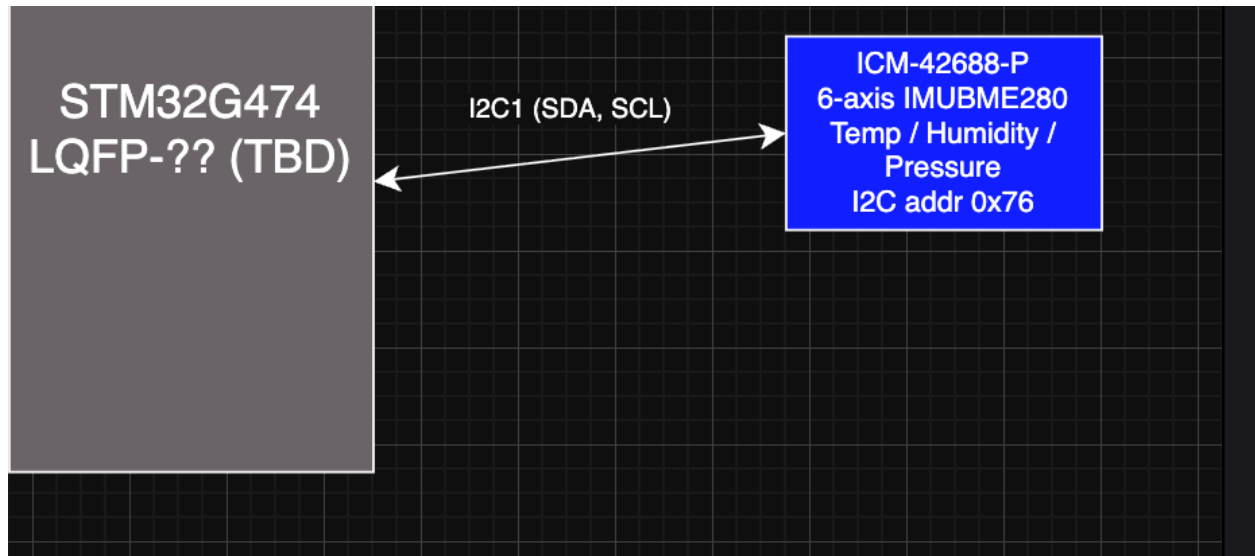
No chip select. Instead, every device has a unique 7-bit address baked into the chip. MCU sends the address on the bus; only the matching device responds. BME280 = 0x76 or 0x77 depending on how its SDO pin is wired.

Multi-drop. You can hang many devices on the same 2 wires — they all share SDA and SCL.

Add another I2C sensor later? Just connect it to the same lines.

Slower. Standard mode = 100 kHz, Fast mode = 400 kHz, Fast-mode Plus = 1 MHz. SPI runs at 10+ MHz easily.

Needs pull-up resistors on SDA and SCL (typically 4.7kΩ to 3.3V). The bus is "open-drain" — devices can only pull the lines low; the resistors pull them high when nobody's talking. Without pull-ups, I2C doesn't work. Common beginner mistake.



CAN Section Notes

CAN bus (general)

- 2-wire differential bus: CANH and CANL
- Used in cars, drones, satellites, industrial robots — Anduril, SpaceX, Tesla all use it
- Multi-drop: many nodes share the same 2 wires, each with a unique message ID
- Robust against electrical noise (differential signaling, like USB D+/D-)
- Speeds: classic CAN up to 1 Mbps, CAN FD ("Flexible Data-rate") up to 5–8 Mbps

TCAN334 (the transceiver)

- Translates between MCU logic (3.3V single-ended) and CAN bus (differential ~2.5V common mode)
- MCU side: TX (out), RX (in) — connects to MCU's FDCAN1 peripheral
- Bus side: CANH, CANL — go out to the connector
- Has built-in ESD protection on the bus pins (CAN connectors are external = ESD-exposed, same logic as USB)
- Powered from 3.3V rail

CAN Connector

- 3 pins: CANH, CANL, GND
- Screw terminal (or DB9) — easy to wire field connections to other CAN nodes
- GND is required as a common reference between nodes

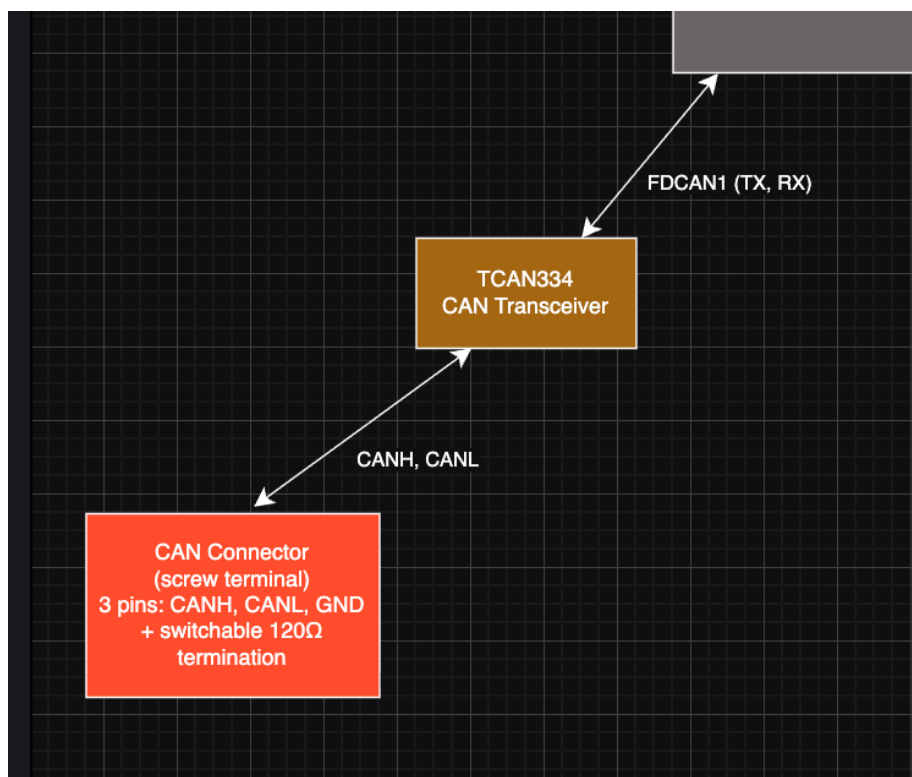
120Ω termination (switchable)

- CAN buses need a 120Ω resistor at *each end* of the bus to absorb signal reflections

- Without termination, signals bounce back and corrupt data — classic transmission-line problem
- A node in the *middle* of the bus does NOT terminate (would create wrong impedance)
- A node at an *end* of the bus DOES terminate
- Your board doesn't know where it'll be on the bus, so make termination switchable (jumper or DIP switch) — user chooses based on physical location

FDCAN1 (the MCU peripheral)

- The G474 has two FDCAN controllers (FDCAN1, FDCAN2). Using FDCAN1.
- "FD" = Flexible Data-rate — supports both classic CAN and the faster CAN FD variant
- Generates the bit timing, handles framing, error detection, arbitration



SWD (Serial Wire Debug)

ARM standard for programming and debugging Cortex-M chips

2 signal pins: SWDIO (bidirectional data), SWCLK (clock)

Plus GND, 3.3V (powers the debugger), NRST (reset)

You plug an ST-Link or J-Link debugger into this header to flash firmware and step through code
Without this header you can't program the board

How it works in practice:

You build your firmware on your laptop → produces a .elf or .bin file

You plug an ST-Link debugger (~\$10 clone, or a Nucleo board can act as one) into the SWD header

The other end of the ST-Link plugs into your laptop via USB
Software on your laptop (OpenOCD, STM32CubeProgrammer) talks through the ST-Link →
through the SWD pins → into the MCU's debug core
Firmware gets written to flash. Done.

What "debug" means here, beyond flashing:

Breakpoints — pause execution at a specific line of code
Step through code one line at a time
Inspect variables in RAM while the chip is running
Read/write registers directly to test peripherals
Halt and resume the CPU
Watch live variables update as the code runs

All of this happens through those same 2 signal wires (SWDIO + SWCLK) — they're a
bidirectional debug protocol, not just "programming."

Why each pin on the header:

SWDIO — bidirectional data
SWCLK — clock from debugger
GND — common reference
3.3V — lets the debugger sense your board's voltage level (so it talks at the right logic level)
NRST — lets the debugger force-reset the MCU if firmware crashes

Without SWD, you can't develop. USB DFU is a fallback for flashing only, not debugging. Real
embedded work means SWD.

UART (Universal Asynchronous Receiver/Transmitter)

Old-school serial port — **TX, RX, GND**

Used for printf-style debug logging when USB isn't working yet
Connect to a USB-to-UART dongle (~\$5) → see logs in any terminal program
Backup channel: if USB enumeration breaks or the bootloader hangs, UART still works

Purpose: print debug messages from the MCU to your laptop.

When code runs on a microcontroller, you can't `print("got here")` and see it on a screen
— the MCU has no screen. You need a way to send text out so you can see what your code is
doing. UART is the simplest way.

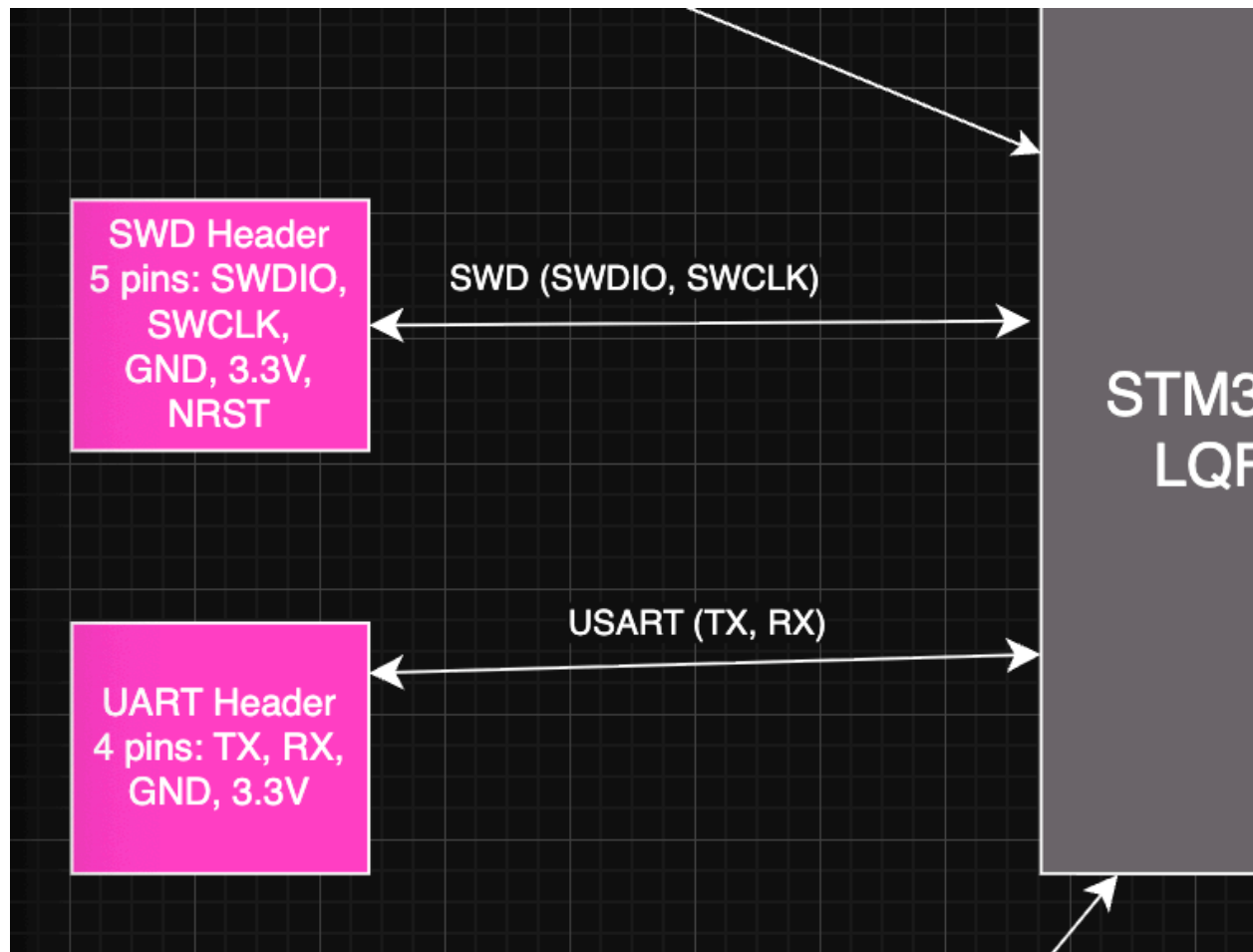
How it works in practice:

1. In your firmware, you initialize the UART peripheral (set baud rate, e.g. 115200)
2. You call `printf("temperature = %d\n", temp)` — firmware HAL routes that
through the UART TX pin
3. UART pin sends bits at 115200 baud → through the header → into a USB-to-UART
dongle (FTDI, CP2102, ~\$5)

4. The dongle shows up on your laptop as a COM port (Windows) or `/dev/ttyUSB0` (Linux/Mac)
5. You open a terminal program (PuTTY, minicom, screen) → see the text scroll by in real time

Why you need this when SWD already exists:

- SWD debugging is *invasive* — when you set a breakpoint, you halt the CPU. Some bugs only happen at full speed (timing, ISRs, real-time stuff). Halting changes the behavior, so you can't catch them.
- UART is *non-invasive* — the MCU keeps running, just spits out text. Perfect for "what's happening during this 50 ms window."
- UART works without a debugger plugged in. Useful for field debugging when the board is deployed.
- UART is your fallback when something is fundamentally broken — e.g., the USB stack didn't enumerate, or the firmware is hung in a fault handler. UART can be made to work from literally the first instruction of `main()`.



GPIO (General-Purpose Input/Output)

Generic digital pins on the MCU. Software controls whether each pin is an input or output.

Output mode: firmware writes 1 or 0 → pin drives 3.3V or 0V. Used for LEDs, enabling chips, chip-select lines.

Input mode: firmware reads the pin → returns 1 if voltage is high, 0 if low. Used for buttons, switches, status lines from other chips.

Open-drain vs push-pull: push-pull actively drives both high and low (normal output).

Open-drain only pulls low; needs an external pull-up to go high (used for I2C, shared signals).

Internal pull-up/pull-down: every STM32 GPIO has optional internal resistors (~40kΩ) you enable in firmware. Saves you from soldering an external resistor for a button.

Alternate function: the same physical pin can be repurposed as SPI clock, UART TX, timer output, etc. CubeMX shows you the alternate-function table — that's the pin-planning step.

Interrupts: a GPIO input can trigger an interrupt on rising/falling edge. Used for "wake me up when the button is pressed" without polling.

For your board: 4 pins as push-pull outputs (LEDs), 1 pin as input with internal pull-up (button).

Total 5 GPIO

